



ORACLE
NetSuite

Bank Connectivity Plug-in



2025.2

September 3, 2025



Copyright © 2005, 2025, Oracle and/or its affiliates.

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software, software documentation, data (as defined in the Federal Acquisition Regulation), or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software," "commercial computer software documentation," or "limited rights data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle®, Java, and MySQL are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

If this document is in public or private pre-General Availability status:

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

If this document is in private pre-General Availability status:

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, timing, and pricing of any features or functionality described in this document may change and remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Documentation Accessibility

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at <https://www.oracle.com/corporate/accessibility>.

Access to Oracle Support

Oracle customers that have purchased support have access to electronic support through My Oracle Support. For information, visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=info> or visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=trs> if you are hearing impaired.

Sample Code

Oracle may provide sample code in SuiteAnswers, the Help Center, User Guides, or elsewhere through help links. All such sample code is provided "as is" and "as available", for use only with an authorized NetSuite Service account, and is made available as a SuiteCloud Technology subject to the SuiteCloud Terms of Service at www.netsuite.com/tos.

Oracle may modify or remove sample code at any time without notice.

No Excessive Use of the Service

As the Service is a multi-tenant service offering on shared databases, Customer may not use the Service in excess of limits or thresholds that Oracle considers commercially reasonable for the Service. If Oracle reasonably concludes that a Customer's use is excessive and/or will cause immediate or ongoing performance issues for one or more of Oracle's other customers, Oracle may slow down or throttle Customer's excess use until such time that Customer's use stays within reasonable limits. If Customer's particular usage pattern requires a higher limit or threshold, then the Customer should procure a subscription to the Service that accommodates a higher limit and/or threshold that more effectively aligns with the Customer's actual usage pattern.

Beta Features

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software" or "commercial computer software documentation" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle and Java are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, and timing of any features or functionality described in this document remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Table of Contents

Bank Connectivity Plug-in Interface Overview	1
Limitations of Using the Bank Connectivity Plug-in	2
Developing a Bank Connectivity Plug-in	2
Creating the Bank Connectivity Plug-in Script File	2
Obtaining the Host Key	3
Administering a Bank Connectivity Plug-in	4
Enabling Features for a Bank Connectivity Plug-in	4
Adding the Plug-in Implementation	4
Defining the Configuration Requirements	5
Activating the Bank Connectivity Plug-in	6
Testing the Bank Connectivity Plug-in	6
Manually Running the Bank Connectivity Plug-in	6
Bank Connectivity Plug-in Interface Definition	7
getRequiredConfigurationFields (context)	8
downloadPreviousDayBankStatementFile (context)	8
Object Types	10
BankAccount	10
BankConnectivityPluginConfiguration	11
RequiredConfigurationFieldsInput	11
DownloadPreviousDayBankStatementFileInput	12
RequiredConfigurationFieldsOutput	13
DownloadPreviousDayBankStatementFileOutput	14

Bank Connectivity Plug-in Interface Overview

Warning: The Bank Connectivity Plug-in is deprecated as of NetSuite 2020.1. This means that the plug-in is still functional, but is no longer supported. To directly connect to a financial institution and automate bank data imports, use the Financial Institution Connectivity Plug-in. For details, see .

Note: The Bank Connectivity Plug-in uses SuiteScript 2.0. Your entry point scripts and supporting library scripts must also use SuiteScript 2.0. For information, see the help topics [SuiteScript 2.x Script Creation Process](#) and [SuiteScript 2.x](#).

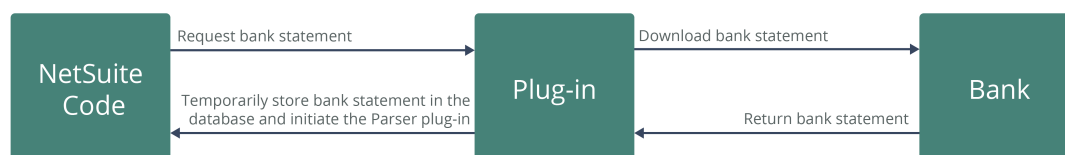
Use the Bank Connectivity Plug-in to make secure electronic connections to financial institutions, for inbound transmissions, through NetSuite. Provided that your financial institution can generate a bank statement file to a specific location on a server, the plug-in can get the file and then upload it to NetSuite for you.

Inbound transmission enables you to automate cash balance reporting and schedule account statement downloads.

After the administrator installs and implements the plug-in, users can define specific configuration requirements. For example, a plug-in using Secure File Transfer Protocol (SFTP) would need authentication information and the location of the SFTP server.

The plug-in is capable of getting the results immediately when it makes a request for end-of-day bank statements. The plug-in formats the bank statement file and then sends it to NetSuite. After the import is complete, continue with other stages of the Cash Management workflow, such as transaction matching and reconciliation. There may be cases where NetSuite requires user input for bank authentication. NetSuite saves credentials securely.

The following diagram shows the synchronous workflow for the `requestPreviousDayBankStatement` function:



The Bank Connectivity Plug-in works in conjunction with the Bank Statement Parser Plug-in. Therefore, the Bank Connectivity Plug-in supports any format that has a Bank Statement Parser Plug-in.

The sample plug-in (SFTP plug-in) provides Host-to-Host (direct) Connectivity using SFTP, which is the supported communication protocol. NetSuite supports any created plug-in using the SFTP protocol.

For more information about the Bank Connectivity Plug-in, see the following topics:

NetSuite Role	For more information, see ...
All roles	Limitations of Using the Bank Connectivity Plug-in
Developer	Developing a Bank Connectivity Plug-in

NetSuite Role	For more information, see ...
Administrator	Administering a Shipping Partner Plug-in Implementation

Limitations of Using the Bank Connectivity Plug-in

When using the Bank Connectivity Plug-in, keep in mind the following limitations:

- To connect to a financial institution to get bank statements, you have to manually trigger the sFTP connection. For details, see the help topic [Automating Imports for Bank Reconciliation](#).
- The plug-in does not allow you to add IP addresses to an allowlist.
- You cannot retrieve ZIP files.

Developing a Bank Connectivity Plug-in

To develop a Bank Connectivity Plug-in, complete the following steps:

- [Step 4. Create the Plug-in Implementation Script File](#)
- [Obtaining the Host Key](#)

Creating the Bank Connectivity Plug-in Script File

You must implement each Bank Connectivity Plug-in interface function in a JavaScript file (with a .js extension) to define the behavior of the plug-in implementation. You can use the SuiteCloud IDE or another JavaScript IDE or text editor to create the plug-in script file.

The following sample implements a basic sFTP connection using the 'N/sftp' SuiteScript module in the plug-in implementation script file:

Note: The 'N/sftp' SuiteScript module supports regular expressions as of 2019.2. For details, see the help topic [N/sftp Module](#).

```

1  /**
2   * @NApiVersion 2.0
3   * @NScriptType bankConnectivityPlugin
4   */
5  define(['N/sftp', 'N/file'],
6    function(sftp, file) {
7    return {
8      getRequiredConfigurationFields: function (context) {
9        //Connection details
10       context.output.addConfigurationField({type:"TEXT", label:"URL", dataName:"url"});
11       context.output.addConfigurationField({type:"TEXT", label:"User name", dataName:"username"});
12       context.output.addConfigurationField({type:"PASSWORD", label:"Password", dataName:"password"});
13       context.output.addConfigurationField({type:"TEXT", label:"Host Key", dataName:"hostKey"});
14       context.output.addConfigurationField({type:"TEXT", label:"Host Key Type", dataName:"hostKeyType"});
15       context.output.addConfigurationField({type:"TEXT", label:"Port", dataName:"port"});
16
17       //File download details
18       context.output.addConfigurationField({type:"TEXT", label:"Download Filename", dataName:"filename"});
19       context.output.addConfigurationField({type:"TEXT", label:"Download Directory", dataName:"directory"});
20       context.output.addConfigurationField({type:"TEXT", label:"Download Timeout", dataName:"timeout"});
21       context.output.addConfigurationField({type:"TEXT", label:"Bank statement format", dataName:"format"});
22     },

```

```

23     downloadPreviousDayBankStatementFile: function (context) {
24         var config = context.input.pluginConfiguration;
25         var connection = sftp.createConnection({
26             url: config.getConfigField({fieldName:"url"}),
27             passwordGuid: config.getConfigField({fieldName:"password"}),
28             hostKey: config.getConfigField({fieldName:"hostKey"}),
29             hostKeyType: config.getConfigField({fieldName:"hostKeyType"}),
30             username: config.getConfigField({fieldName:"username"}),
31             port: parseInt(config.getConfigField({fieldName:"port"}))
32         });
33         var downloadedFile = connection.download({
34             filename: config.getConfigField({fieldName:"filename"}),
35             directory: config.getConfigField({fieldName:"directory"}),
36             timeout: config.getConfigField({fieldName:"timeout"})
37         });
38         context.output.saveBankStatementFile({file: downloadedFile, bankStatementFormat: config.getConfigFieldVal
ue({fieldName:"format"}}));
39     },
40 },
41 },
42 );

```

For details and breakdowns of the sample, see [Bank Connectivity Plug-in Interface Definition](#).

The following table describes the interface functions:

Function	Description
getRequiredConfigurationFields (context)	Define the configuration requirements for your bank connectivity implementation. For example, a specific bank may require a secret user name and password to connect. This function is called when you open the plug-in configuration page. (used to define the fields you see on that page)
downloadPreviousDayBankStatementFile (context)	Request a statement from the bank. When you request a statement from NetSuite, the plug-in calls the function which makes the download request to the bank. The bank sends the statement back to the plug-in, then the plug-in formats and sends the statement to NetSuite. Continue with other stages of the Cash Management workflow, like transaction matching.

Rules and Guidelines for Creating a Plug-in Implementation Script File

Use the following rules and guidelines when creating the plug-in implementation script file:

- The plug-in script file can have any name, as long as it contains an implementation of each of the interface functions.
- If you want to create utility files with helper functions to use with the plug-in implementation script file, you can include those files when you create the plug-in implementation in NetSuite. See [Step 5. Add and Activate the Plug-in Implementation](#) and [Activating the Bank Connectivity Plug-in](#).
- Due to limitations with how strings are handled in NetSuite, you cannot use the JavaScript case statement in the plug-in implementation script file.
- The plug-in allows up to 1000 usage units.

Obtaining the Host Key

The host key can be obtained using OpenSSH's ssh-keyscan tool:


```
1 | ssh-keyscan -t <hostKeyType> -p <port> <hostDomain>
```

Each bank has different conventions for file location and naming. Extend this sample to match the way your bank provides daily bank statements. Use the following set of basic configuration fields to download a specific file from the sFTP server:

- Download File Name – The specific file to download from the server.
- Download Directory – The folder location on the server (path relative to the user's home directory) where the file resides.
- Download Timeout – The number of seconds to allow the file to download.
- Bank Statement Format — Any format that has a Bank Statement Parser Plug-in is supported.

If any of these configuration fields are not required for your sFTP connection to a bank, you can remove them or change the values as required.

Administering a Bank Connectivity Plug-in

To install and set up a Bank Connectivity Plug-in, complete the following steps:

- [Step 1. Enable Features for Development](#)
- [Step 5. Add and Activate the Plug-in Implementation](#)
- [Defining the Configuration Requirements](#)
- [Activating the Bank Connectivity Plug-in](#)
- [Testing the Bank Connectivity Plug-in](#)
- [Manually Running the Bank Connectivity Plug-in](#)

Enabling Features for a Bank Connectivity Plug-in

To begin development of a Bank Connectivity Plug-in, make sure that the Server SuiteScript feature is enabled on the development account.

To enable features for the plug-in:

1. Choose Setup > Company > Enable Features.
2. On the **SuiteCloud** subtab, make sure **Server SuiteScript** is checked. If necessary, check the box and agree to the Terms of Service.
3. Click **Save**.

Adding the Plug-in Implementation

After creating the plug-in script file, upload and implement the file in NetSuite.

To add the plug-in implementation:

1. In NetSuite, go to Customization > Plug-ins > Plug-in Implementations > New.
NetSuite displays the Upload Plug-in Implementation page.
2. For the **Script File**, click the plus icon.

3. Click **Choose File**, then select the JavaScript file.
4. Click **Save**.
5. Click **Create Plug-in Implementation**.
6. On the **Plug-in Implementation** page, enter the following information:

Note: There is no need to select the plug-in type or the implementation of the plug-in. Because the plug-in uses SuiteScript 2.0, these fields are sourced from the plug-in script file.

Option	Description
Name	User-friendly name for the implementation. This name appears in the plug-in implementation list.
ID	Internal ID for the implementation for use in scripting. If you do not provide an ID, NetSuite will provide one for you after you click Save . As a developer, when you write code that uses this implementation, you will reference the implementation using this ID.
Status	Current status for the implementation. Choose Testing to have the implementation accessible to the owner of the implementation. Choose Released to have the implementation accessible in a production environment
Log Level	Logging level you want for the script. Select Debug, Audit, Error, or Emergency. The log information appears on the Execution Log subtab for the implementation after you create it. Go to Customization > Plug-ins > Plug-in Implementations, select your implementation, and then click the Execution Log subtab.
Description	Optional description of the implementation.
Owner	User account that owns the implementation. Default is the name of the current user

7. On the **Unhandled Errors** subtab, select which individual(s) will be notified if script errors occur. By default the **Notify Script Owner** box is checked.
To enter multiple email addresses in the **Notify Emails** box, separate email addresses with a semi-colon.
8. Click **Save**.

Defining the Configuration Requirements

After the administrator installs and implements the plug-in, you can define specific configuration requirements. The options available depend on the set-up configuration in the JavaScript file.

You may require a secret username or password to connect to the bank.

To define configuration requirements:

1. Go to Customization > Plug-ins > Plug-in Implementations.
NetSuite displays the list page.
2. For your plug-in implementation, click **Edit**.
NetSuite displays the implementation page for your plug-in.
3. Click **Configure**.
NetSuite dynamically generates the form from the plug-in implementation.
4. If defined by the plug-in, you may need to set:
 - URL for the connecting bank

- user name and password
- host key and key type
- port
- file download information

5. Click **Save**.

NetSuite saves your configuration for the plug-in.

Activating the Bank Connectivity Plug-in

To use the plug-in, activate your implementation. For more information, see the help topic [Managing Custom Plug-in Implementations](#).

To activate the plug-in:

1. To activate the plug-in implementation, go to Customization > Plug-ins > Manage Plug-ins.
2. Under **Bank Connectivity**, check the box next to name of your implementation.
3. Click **Save**.

Testing the Bank Connectivity Plug-in

To test the plug-in, run the plug-in and then go to Transactions > Bank > Import Online Banking Data > List to view the downloaded statement.

Manually Running the Bank Connectivity Plug-in

If you want to run the plug-in, you can manually invoke the plug-in to download and import bank statement transactions. For details, see the help topic [Automating Imports for Bank Reconciliation](#).

Bank Connectivity Plug-in Interface Definition

 **Warning:** The Bank Connectivity API is deprecated as of NetSuite 2020.1. To use a connectivity plug-in that can directly connect you to a financial institution and automate bank data imports, use the Financial Institution Connectivity API. For details, see the help topic [Financial Institution Connectivity Plug-in Interface Definition](#).

The plug-in interface includes the following functions.

 **Important:** You cannot change these function signatures and their return types in a Bank Connectivity Plug-in implementation.

Function	Description
getRequiredConfigurationFields (context)	Define the configuration requirements for your bank connectivity implementation. For example, a specific bank may require a secret user name and password to connect. This function is called when you open the plug-in configuration page.
downloadPreviousDayBankStatementFile (context)	Request a statement from the bank. When you request a statement from NetSuite, the plug-in calls the function which makes the download request to the bank. The bank sends the statement back to the plug-in, then the plug-in formats and sends the statement to NetSuite. Continue with other stages of the Cash Management workflow, like transaction matching.

The plug-in interface includes the following objects.

Object	Description
BankAccount	To provide the plug-in with required information, this object contains a small subset of data from a bank account record in NetSuite.
BankConnectivityPluginConfiguration	This object contains all the user-supplied configuration data (field values) for this plug-in.
RequiredConfigurationFieldsInput	The getRequiredConfigurationFields (context) function uses this object.
DownloadPreviousDayBankStatementFileInput	The downloadPreviousDayBankStatementFile (context) function uses this object. When the bank sends a requested statement back to the plug-in, the plug-in formats and sends the statement to NetSuite.
RequiredConfigurationFieldsOutput	The getRequiredConfigurationFields (context) function uses this object. The plug-in must use this object to provide any configuration fields it requires the user to complete. Based on this configured output, the required fields populate the Bank Connectivity Plug-In Configuration page in NetSuite.

Object	Description
DownloadPreviousDayBankStatementFileOutput	The downloadPreviousDayBankStatementFile (context) function uses this object. When the bank sends a requested statement back to the plug-in, the plug-in formats and sends the statement to NetSuite.

getRequiredConfigurationFields (context)

Function Declaration	<code>void getRequiredConfigurationFields(RequiredConfigurationFieldsContext context)</code>
Type	Function
Description	Provides a list of configuration fields, which the user must complete to use the other plug-in functions. The plug-in provides all functions with the user-supplied values for these fields in the input property of the respective context for the function. To define each configuration field line, see addConfigurationField .
Returns	void
Input Parameters	RequiredConfigurationFieldsContext

Example

```

1  getRequiredConfigurationFields: function (context) {
2      //Connection details
3      context.output.addConfigurationField({type:"TEXT", label:"URL", dataName:"url"});
4      context.output.addConfigurationField({type:"TEXT", label:"Username", dataName:"username"});
5      context.output.addConfigurationField({type:"PASSWORD", label:"Password", dataName:"password"});
6      context.output.addConfigurationField({type:"TEXT", label:"Host Key", dataName:"hostKey"});
7      context.output.addConfigurationField({type:"TEXT", label:"Host Key Type", dataName:"hostKeyType"});
8      context.output.addConfigurationField({type:"TEXT", label:"Port", dataName:"port"});
9
10     //File download details
11     context.output.addConfigurationField({type:"TEXT", label:"Download Filename", dataName:"filename"});
12     context.output.addConfigurationField({type:"TEXT", label:"Download Directory", dataName:"directory"});
13     context.output.addConfigurationField({type:"TEXT", label:"Download Timeout", dataName:"timeout"});
14     context.output.addConfigurationField({type:"TEXT", label:"Bank statement format", dataName:"format"});
15 }

```

For more details, see [addConfigurationField](#).

downloadPreviousDayBankStatementFile (context)

Function Declaration	<code>void downloadPreviousDayBankStatementFile(DownloadPreviousDayBankStatementFileContext context)</code>
Type	Function
Description	Requests the previous day's statement from the bank. When you request a statement from NetSuite, the plug-in calls the function which triggers the download request to the bank. The bank sends the statement back to the plug-in, then the plug-in formats and sends the statement to NetSuite.
Returns	void
Input Parameters	DownloadPreviousDayBankStatementFileContext

Example

The following is an example of the whole `downloadPreviousDayBankStatementFile` function:

```
1 | downloadPreviousDayBankStatementFile: function (context) {
2 |   var config = context.input.pluginConfiguration;
3 |   var connection = sftp.createConnection({
4 |     url: config.getConfigurationFieldValue({fieldName:"url"}),
5 |     passwordGuid: config.getConfigurationFieldValue({fieldName:"password"}),
6 |     hostKey: config.getConfigurationFieldValue({fieldName:"hostKey"}),
7 |     hostKeyType: config.getConfigurationFieldValue({fieldName:"hostKeyType"}),
8 |     username: config.getConfigurationFieldValue({fieldName:"username"}),
9 |     port: parseInt(config.getConfigurationFieldValue({fieldName:"port"}))
10 |   });
11 |   var downloadedFile = connection.download({
12 |     filename: config.getConfigurationFieldValue({fieldName:"filename"}),
13 |     directory: config.getConfigurationFieldValue({fieldName:"directory"}),
14 |     timeout: parseInt(config.getConfigurationFieldValue({fieldName:"timeout"}))
15 |   });
16 |   context.output.saveBankStatementFile({file: downloadedFile, bankStatementFormat: config.getConfigurationFieldValue({fieldName:"format"})});
17 | }
```

As in this example, first create a convenience variable to easily reference the plug-in configuration values, which are provided as input to the function:

```
1 | var config = context.input.pluginConfiguration;
```

Create the SFTP connection by using the SuiteScript 'N/sftp' module.

Provide the configuration values for the fields from the defined [getRequiredConfigurationFields \(context\)](#). The `fieldName` property in `config.getConfigurationFieldValue` maps directly to the `dataName` property in the `getRequiredConfigurationFields` definition. To define each configuration field line in `getRequiredConfigurationFields`, see [addConfigurationField](#).

```
1 | var connection = sftp.createConnection({
2 |   url: config.getConfigurationFieldValue({fieldName:"url"}),
3 |   passwordGuid: config.getConfigurationFieldValue({fieldName:"password"}),
4 |   hostKey: config.getConfigurationFieldValue({fieldName:"hostKey"}),
5 |   hostKeyType: config.getConfigurationFieldValue({fieldName:"hostKeyType"}),
6 |   username: config.getConfigurationFieldValue({fieldName:"username"}),
7 |   port: parseInt(config.getConfigurationFieldValue({fieldName:"port"}))
8 | });
```

Using the set configuration field values for the file name and directory, download the file to the NetSuite server.

Based on your bank's conventions, you may need to create functions to determine the file name or directory. For example, your bank may require the file name to include the date.

In our sample, there is a single directory containing a file that regenerates with new content once a day.

```
1 | var downloadedFile = connection.download({
2 |   filename: config.getConfigurationFieldValue({fieldName:"filename"}),
3 |   directory: config.getConfigurationFieldValue({fieldName:"directory"}),
4 |   timeout: parseInt(config.getConfigurationFieldValue({fieldName:"timeout"}))
5 | });
```

Lastly, provide the downloaded file back to the NetSuite system. Once in NetSuite, the administrator can import the file and automatic matching runs on the statement transactions.

```
1 | context.output.saveBankStatementFile({file: downloadedFile, bankStatementFormat: config.getConfigurationFieldValue({fieldName:"format"})});
```

Object Types

This section describes the data objects for the plug-in, which the functions use. These objects modify the specified data and include the following types:

- [BankAccount](#)
- [BankConnectivityPluginConfiguration](#)
- [RequiredConfigurationFieldsInput](#)
- [DownloadPreviousDayBankStatementFileInput](#)
- [RequiredConfigurationFieldsOutput](#)
- [DownloadPreviousDayBankStatementFileOutput](#)

BankAccount

Type	Interface object
Description	To provide the plug-in with required information, this object contains a small subset of data from a bank account record in NetSuite.
Object Functions	■ getAccountNumber() ■ getId()
Parent Object(s)	N/A
Child Object(s)	N/A

getAccountNumber()

Function Declaration	string getAccountNumber()
Type	Object function
Description	This function returns the account number of the bank account record in NetSuite.
Returns	string
Input Parameters	None
Parent object	BankAccount

getId()

Function Declaration	string getId()
Type	Object function
Description	This function returns the internal ID of the bank account record in NetSuite.
Returns	string
Input Parameters	None
Parent object	BankAccount

BankConnectivityPluginConfiguration

Type	Interface object
Description	This object contains all the user-supplied configuration data (field values) for this plug-in.
Object Functions	getConfigurationFieldValue
Parent Object(s)	DownloadPreviousDayBankStatementFileInput
Child Object(s)	N/A

getConfigurationFieldValue

Function Declaration	string getConfigurationFieldValue(string dataName)
Type	Object function
Description	NetSuite uses this function to return the user-supplied configuration value for a required configuration field. This function retrieves the data using the defined keys in the addConfigurationField function's dataName parameter.
Returns	string – the user-supplied configuration value.
Input Parameters	dataName – the name of the key used to access the field when accessing the BankConnectivityPluginConfiguration input.
Parent object	BankConnectivityPluginConfiguration

Example

```
1 | context.output.saveBankStatementFile({file: downloadedFile, bankStatementFormat: config.getConfigurationFieldValue({field
   | Name:"format"})});
```

RequiredConfigurationFieldsInput

Type	Interface object
Description	The getRequiredConfigurationFields (context) function uses this object.
Object Functions	■ getLocale ■ getPluginConfiguration
Parent Object(s)	N/A
Child Object(s)	N/A

getLocale

Function Declaration	string getLocale()
Type	Object function

Description	NetSuite uses this object function to return the location of the current user session (for example, 'en_US').
Returns	string – the locale string for the current session.
Input Parameters	None
Parent object	■ DownloadPreviousDayBankStatementFileInput ■ RequiredConfigurationFieldsInput

getPluginConfiguration

Function Declaration	<code>BankConnectivityPluginConfiguration getPluginConfiguration()</code>
Type	Object function
Description	NetSuite uses this object function to return the user-supplied configuration for the required configuration fields, which the getRequiredConfigurationFields (context) function defines.
Returns	BankConnectivityPluginConfiguration – the user-supplied configuration, if it exists. If the user does not supply any details for the configuration, this function returns null.
Input Parameters	None
Parent object	■ DownloadPreviousDayBankStatementFileInput ■ RequiredConfigurationFieldsInput

DownloadPreviousDayBankStatementFileInput

Type	Interface object
Description	The downloadPreviousDayBankStatementFile (context) function uses this object. When you request a statement from NetSuite, the plug-in calls the function which triggers the download request to the bank. The bank sends the statement back to the plug-in.
Object Functions	■ getLocale ■ getPluginConfiguration ■ getSecureFileCabinetFolderId()
Parent Object(s)	N/A
Child Object(s)	BankConnectivityPluginConfiguration

getSecureFileCabinetFolderId()

Function Declaration	<code>int getSecureFileCabinetFolderId()</code>
Type	Object function
Description	NetSuite uses this object function to return the ID of the folder to which the bank statement file must be saved.
Returns	int – the ID of the folder.

Input Parameters	None
Parent object	DownloadPreviousDayBankStatementFileInput

RequiredConfigurationFieldsOutput

Type	Interface object
Description	The getRequiredConfigurationFields (context) function uses this object. The plug-in must use this object to provide any configuration fields it requires the user to complete. Based on this configured output, the required fields populate the Bank Connectivity Plug-In Configuration page in NetSuite.
Object Functions	■ addConfigurationField ■ RequiredConfigurationFieldsContainer
Parent Object(s)	N/A
Child Object(s)	N/A

addConfigurationField

Function Declaration	void addConfigurationField(string type, string label, string dataName, boolean required)
Type	Object function
Description	Add a required configuration field for this plug-in to be available on the Bank Connectivity Plug-In Configuration page in NetSuite. Since SuiteScript cannot provide type-safety for the field type, choose the type parameter from one of the following string values: ■ PASSWORD - For the mandatory password field, which provides passwords to SuiteScript modules, including the 'N/sftp' module. The password is securely stored as a token that these modules will be able to exchange for the actual password. If you need to store a password in plain-text for whatever reason, all of the configuration field data is stored securely ■ TEXT - A plain text box ■ TEXTAREA - A text area for multi-line content
Returns	void
Input Parameters	type – A string chosen from the list of field types (PASSWORD, TEXT, or TEXTAREA). label – The label to display on the form.  Note: The label parameter can be localized by taking into account the input's getLocale function. Otherwise, this output object should be deterministic. dataName – The name of the key used to access the field when accessing the BankConnectivityPluginConfiguration input. required – Specifies whether a field is required. If true, the field is mandatory. The default is false.
Parent object	■ RequiredConfigurationFieldsOutput ■ DownloadPreviousDayBankStatementFileInput

Example

```

1   getRequiredConfigurationFields: function (context) {
2   //Connection details
3   context.output.addConfigurationField({type:"TEXT", label:"URL", required:true, dataName:"url"});
4   context.output.addConfigurationField({type:"TEXT", label:"User name", dataName:"username"});
5   context.output.addConfigurationField({type:"PASSWORD", label:"Password", dataName:"password"});
6   context.output.addConfigurationField({type:"TEXT", label:"Host Key", dataName:"hostKey"});
7   context.output.addConfigurationField({type:"TEXT", label:"Host Key Type", dataName:"hostKeyType"});
8   context.output.addConfigurationField({type:"TEXT", label:"Port", dataName:"port"});
9
10  //File download details
11  context.output.addConfigurationField({type:"TEXT", label:"Download Filename", dataName:"filename"});
12  context.output.addConfigurationField({type:"TEXT", label:"Download Directory", dataName:"directory"});
13  context.output.addConfigurationField({type:"TEXT", label:"Download Timeout", dataName:"timeout"});
14  context.output.addConfigurationField({type:"TEXT", label:"Bank statement format", dataName:"format"});
15 },

```

This example mirrors all of the connection details provided by the 'N/sftp' module.

- URL – The URL of the bank's SFTP server.
- User name – The user name to connect to the bank's SFTP server.
- Password – The password for the user.
- Host Key – The finger print for the bank's SFTP server. For example, a finger print could be "c4:26:ab:cf:a0:15:0a:99:5f:f3:7b:ef:3b:19:d8:96". See [Obtaining the Host Key](#).
- Host Key Type – The type of host key provided by the bank's server. The host key type is usually RSA.
- Port – The port the SFTP server is using, which is typically 22 since SFTP is over SSH.

RequiredConfigurationFieldsContainer

Function Declaration	RequiredConfigurationFieldsContainer requiredConfigurationFieldsContainer()
Type	Object function
Description	NetSuite uses this function to extract the configurations field values that the plug-in returns.
Returns	RequiredConfigurationFieldsContainer
Input Parameters	None
Parent object	RequiredConfigurationFieldsOutput

DownloadPreviousDayBankStatementFileOutput

Type	Interface object
Description	The downloadPreviousDayBankStatementFile (context) function uses this object. When the bank sends a requested statement back to the plug-in, the plug-in formats and sends the statement to NetSuite.
Object Function	■ saveBankStatementFile
Parent Object(s)	N/A
Child Object(s)	N/A

saveBankStatementFile

Function Declaration	<code>void saveBankStatementFile(File file, string bankStatementFormat)</code>
Type	Object function
Description	NetSuite uses this function to save the bank statement file returned by the plug-in.
Returns	void
Input Parameters	<code>file</code> <code>bankStatementFormat</code>
Parent object	DownloadPreviousDayBankStatementFileOutput